

Extra notes:

Priority Inversion:

one high and one low using the same resource. The other task should be in the medium.

Interweaving

↳ for minimum value

↳ 1st does its job

↳ 2nd runs everything except what loop
everything else finishes everything

↳ 1st does up and stores

↳ 2nd loads, getting ready at 1st up again

↳ 1st finishes very early

↳ 2nd does it and store.

Dining Philosopher

↳ resolves the issue of broken previous dead lock

But, waiting

↳ repeatedly checking a condition

↳ consumes CPU's!

Deadlock!!

↳ single core - 1 thread at a time!

↳ regardless of how many threads the
problem kernel has

mutual exclusion

↳ S=1

wait(S)

// S

signal(S)

K on other sides

S=K

wait(S)

// S

signal(S)